Pure Vision SLAM Engineering in Robotic Edge Computing Systems

Jian Chen

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science in Artificial Intelligence
of the
University of Aberdeen.

Department of Computing Science

2025

Declaration

No portion of the work contained in this document has been submitted in support of an application for a degree or qualification of this or any other university or other institution of learning. All verbatim extracts have been distinguished by quotation marks, and all sources of information have been specifically acknowledged.

Signed:

Date: 2025

Abstract

This thesis presents the engineering implementation of a robotic pure vision SLAM (Simultaneous Localization and Mapping) system based on a stereo camera, designed for edge computing embedded systems. The work focuses on the practical application of visual SLAM techniques in robotics, emphasizing the integration of stereo vision, edge computing, and lightweight embedded systems to achieve efficient and effective mapping, localization, multi-agent (robot and drone) collaboration, and navigation in real-time environments.

In terms of software architecture, the system is built on Ubuntu 22.04 LTS, utilizing ROS2 Humble as the middleware. Key packages include Turtlebot3_bringup, zed-ros2, rtabmap-ros2, uoa_robot_drone, rviz2, and navigation2. The main functionalities encompass a robotic control module, stereo vision image capture module, image format conversion and compression transmission module, visual SLAM algorithm module, localization monitoring and publishing module, automated drone delivery module, navigation module, and visualization module. The system aims to process stereo images in real-time to create an environmental map while tracking the robot's position within that map. The implementation adheres to edge computing principles to ensure low latency and high efficiency, making it suitable for deployment in resource-constrained embedded systems.

The hardware setup includes a Turtlebot3 Waffle Pi robot base (SBC: Raspberry Pi Model 4B, OpenCR: STM32F746ZGT6 / 32-bit ARM Cortex, Battery: MVMOD 3S, 2200 mAh 11.1 V 35C Lipo Battery), an embedded edge computing unit using Jetson Orin Nano (max power 15W, Battery: INIU 65W 20000mAh Power Bank), a ZED 2 stereo camera (resolution set to VGA, frame rate set to 15Hz), a Lenovo Yoga 14S laptop for remote monitoring (Processor: Intel Core i5-1165G7, Memory: 16GB LPDDR4x, Storage: 512GB SSD), and a DJI Tello EDU drone for delivery tasks. The communication between devices is facilitated through a mobile hotspot, utilizing 5GHz WiFi during mapping, localization, and navigation phases, and switching to 2.4GHz WiFi for the delivery phase due to the Tello's compatibility.

The system successfully demonstrates real-time mapping and localization capabilities,

along with automated drone delivery tasks. This project showcases the potential of edge computing in robotic visual SLAM applications and demonstrates the process of engineering mathematics, computer science, AI and other related knowledge into products.

Acknowledgements

I would like to express my sincere gratitude to all those who have supported me throughout this research journey.

First and foremost, I am deeply grateful to my family and my mentor Professor Jiasu Sun from Peking University, whose unwavering support enabled me to make the decision to return to university campus for full-time, systematic study in the field of Artificial Intelligence after years of professional work in industry.

I extend my heartfelt thanks to Professor Dewei Yi at the University of Aberdeen, who not only taught me *Machine Learning* and *Applied Artificial Intelligence* courses, but also inspired me during this dissertation project to transform theoretical knowledge in mathematics, computer science, and AI into practical engineering solutions that enhance real-world productivity. Through this project, I have significantly improved my ability to convert theoretical concepts in robotics, stereo visual SLAM, neural networks, edge computing, embedded systems, and navigation algorithms into concrete products through engineering approaches.

I would also like to acknowledge, in chronological order of instruction, all the professors and lecturers who taught me AI courses and provided me with systematic knowledge in the field of Artificial Intelligence: Professor Felipe Meneguzzi and Dr. Yaji Sripada for *Symbolic AI*; Dr. Mingjun Zhong, who together with Professor Dewei Yi taught *Machine Learning* and *Applied Artificial Intelligence*; Dr. Wei Zhao for *Evaluation of AI Systems*; Dr. Rafael Cardoso and Dr. Leonardo Amado for *Software Agents and Multi-Agent Systems*, and *Knowledge Representation and Reasoning*; Professor Ehud Reiter, who together with Dr. Yaji Sripada taught *Natural Language Generation*; and Dr. Arabella Sinclair for *Data Mining with Deep Learning*.

Looking forward, I am committed to applying AI knowledge through engineering approaches to develop practical products that contribute to human productivity advancement, which I believe is the best way to honor the education and guidance I have received from all these remarkable educators.

Contents

Chapter 1

Introduction

Engineering can enable acdamic papers to truely change the world. When sophisticated mathematical formulations and AI theoretical frameworks are effectively translated into tangible products through engineering approaches, their potential to drive human productivity reaches its fullest expression.

This thesis presents an engineering perspective on the implementation of a robotic pure vision SLAM (Simultaneous Localization and Mapping) system designed for lightweight edge computing embedded systems, which involves:

1. Theoretical foundations (rtabmap and ROS2)
2. Engineering Design
3. Engineering Implementation
4. Engineering Evaluation
5. Conclusions

The hardware utilized includes Turtlebot3 Waffle Pi, ZED 2 stereo camera, Jetson Orin Nano edge computing unit, INIU 65W 20000mAh power bank, and DJI Tello EDU drone.

The main software packages used are Ubuntu 22.04 LTS Server, Ubuntu 22.04 LTS Desktop, JetPack 6.2.1, ROS2 Humble, turtlebot3_bringup, DynamixelSDK, rtabmap, rtabmap_ros, zedros2wrapper, turtlebot3_teleop, DJITelloPy, navigation2, rviz2, rtabmap_viz, rqt.

The conclusion is that pure visual SLAM based on binocular cameras can be applied to edge computing embedded systems with limited computing resources through engineering methods to achieve mapping, positioning, multi-agent (robot and drone) collaboration, and navigation.

Chapter 2

Theoretical Foundations

Simultaneous localization and mapping (SLAM) (**?**) is the capability required by robots to build and update a map of its operating environment and to localize itself in it. A key feature in SLAM is to recognize previously visited locations. This process is also known as loop closure detection, referring to the fact that coming back to a previously visited location makes it possible to associate this location with another one recently visited.

This chapter will introduce the theoretical foundations of the engineering project, which mainly includes the following contents:

- The working principle of RTAB-Map
- The ROS2 working framework

2.1 RTAB-Map Working Principle

Firstly, the first theoretical foundation of this engineering project is introduced: RTAB-Map (Real-Time Appearance-Based Mapping), which is a RGB-D, Stereo and Lidar Graph-Based SLAM approach based on an incremental appearance-based loop closure detector (**?**). In this engineering project, the SLAM method is only based on the stereo camera graph, which is a purely visual SLAM approach. Regarding its principles, this section will focus on the memory management aspects introduced in (**?**).

For global loop closure detection approaches (**?????**), a general way to detect a loop closure is to compare a new location with the ones associated with previously visited locations. If no match is found, then a new location is added. A problem with this approach is that the amount of time required to process new observations increases with the number of locations in the map. If this becomes larger than the acquisition time, a delay is introduced, leading to an obsolete map. In addition, a robot operating in large areas for a long period of time will ultimately build a very large map, making updating and processing the map difficult to achieve in real-time.

The idea of RTAB-Map resides in only using a certain number of locations for loop closure detection so that real-time constraint can be satisfied, while still having access to the locations of the entire map when necessary. When the number of locations in the map makes processing cycle time for finding matches greater than a real-time threshold, this approach transfers locations less likely to cause loop closure detection from the robot's **Working Memory (WM)** to **Long Term Memory (LTM)**, so that they do not take part in the detection of loop closures. However, if a loop closure is detected, neighbor locations can be retrieved and brought back into **WM** to be considered in loop closure detections. This mechanism can be seen as a dynamic way to generate and to dynamically maintain a constant number of locations in **WM**.

In order to keep in **WM** the locations that are more susceptible to be revisited, RTAB-Map evaluates the number of times a location has been matched or consecutively viewed (referred to as **Rehearsal**) to set its weight, making it possible to prioritize the locations seen more often than others and that are more likely to cause loop closures. When transfer occurs, the location with the lowest weight is selected. If there are locations with the same weight, then the oldest one is selected to be transferred to **LTM**.

Similar to (**?**), locations in the **Short Term Memory (STM)** are not used for loop closure detection, to avoid loop closure detection on locations that have just been visited (the last location always looks similar to the most recent ones). **STM** size is fixed based on the robot velocity and the rate at which the locations are acquired. When the number of locations reaches **STM** size limit, the oldest location is moved into **WM**.

Figure 2.1 illustrates the overall loop closure detection process which is explained in details in the following subsections.

2.1.1 Image Location

The bag-of-words approach (**?**) is used to create a signature z_t of an image acquired at time t. An image signature is represented by a set of visual words contained in a visual dictionary incrementally constructed online using a randomized forest of kd-trees (**?**). Speeded-Up Robust Features (SURF) (**?**) are extracted from the image as the visual words. A location L_t is then created with signature z_t, a weight initialized to 0, and a bidirectional link in the graph with the previous location L_{t-1}. The dictionary contains only words of locations in **WM** and **STM**.

2.1.2 Rehearsal

To update the weight of the acquired location, L_t is compared to the ones in **STM** from the most recent ones to the latest, and similarity $\mathfrak{S}$ is evaluated using equation (2.1):

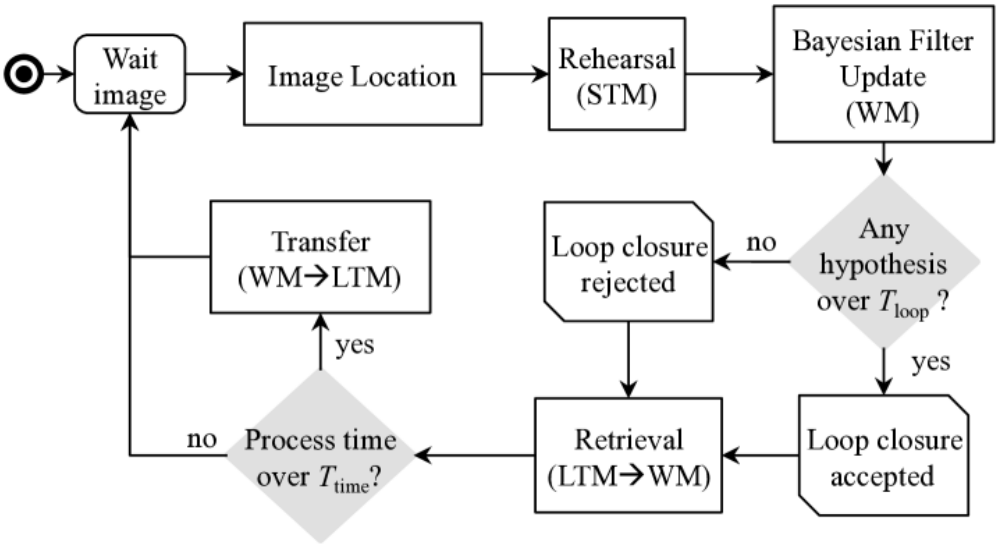


Figure 2.1: Flow chart of RTAB-Map memory management loop closure detection processing cycle.

$$\mathfrak{S}(z_t, z_c) = \begin{cases} \dfrac{N_{\text{pair}}}{N_{z_t}}, & \text{if } N_{z_t} \geq N_{z_c} \\ \dfrac{N_{\text{pair}}}{N_{z_c}}, & \text{if } N_{z_t} < N_{z_c} \end{cases} \tag{2.1}$$

where N_{pair} is the number of matched word pairs between the compared location signatures, and N_{z_t} and N_{z_c} are the total number of words of signature z_t and the compared signature z_c respectively. If $\mathfrak{S}(z_t, z_c)$ is higher than a fixed similarity threshold $T_{\text{rehearsal}}$, L_c is merged into L_t and Rehearsal stops. In Rehearsal, only words of z_c are kept in the merged signature: z_t is cleared and z_c is copied into z_t. To complete the merging process, the weight of L_t is increased by the one of L_c plus one, the neighbor links of L_c are added to L_t, and L_c is deleted from **STM**.

2.1.3 Bayesian Filter Update

The role of the discrete Bayesian filter is to keep track of loop closure hypotheses by estimating the probability that the current location L_t matches one of the already visited locations stored in the **WM**. Let S_t be a random variable representing the states of all loop closure hypotheses at time t. $S_t = i$ denotes the hypothesis that L_t closes a loop with a past location L_i, thus detecting that L_t and L_i represent the same location. $S_t = -1$ denotes the hypothesis that L_t is a new location. The filter estimates the full posterior probability $\mathbf{p}(S_t \mid L^t)$ for all $i = -1, \ldots, t_n$, where t_n is the time index associated with the newest location in the **WM**, expressed by the following equation (2.2):

$$\mathbf{p}(S_t \mid L^t) = \eta \underbrace{\mathbf{p}(L_t \mid S_t)}_{\text{Observation}} \underbrace{\sum_{i=-1}^{t_n} \underbrace{\mathbf{p}(S_t \mid S_{t-1} = i)}_{\text{Transition}} \mathbf{p}(S_{t-1} = i \mid L^{t-1})}_{\text{Belief}} \tag{2.2}$$

where η is a normalization factor and L^t is the set of all locations from L_{-1} to L_t in **WM** and **STM**, thus L^t changes over time as new locations are added and when some locations are retrieved from **LTM** or transferred to **LTM**, deviating from the classical Bayesian filtering where such set is fixed. The observation model $\mathbf{p}(L_t \mid S_t)$ is evaluated using a likelihood function $\mathfrak{L}(S_t \mid L_t)$: the current location L_t is compared using equation (2.1) to locations corresponding to each loop closure state $S_t = j$ where $j = -1, \ldots, t_n$, giving a score $S_j = \mathfrak{S}(z_t, z_j)$. Each score is then normalized using the difference between the score S_j and the standard deviation σ, normalized by the mean μ of all non-null scores, as in equation (2.3)(**?**):

$$\mathbf{p}(L_t \mid S_t = j) = \mathfrak{L}(S_t = j \mid L_t) = \begin{cases} \dfrac{S_j - \sigma}{\mu}, & \text{if } S_j \geq \mu + \sigma \\ 1, & \text{otherwise.} \end{cases} \tag{2.3}$$

For the new location probability $S_t = -1$, the likelihood is evaluated using equation (2.4):

$$\mathbf{p}(L_t \mid S_t = -1) = \mathfrak{L}(S_t = -1 \mid L_t) = \frac{\mu}{\sigma} + 1 \tag{2.4}$$

where the score is relative to μ on σ ratio. If $\mathfrak{L}(S_t = -1 \mid L_t)$ is high, meaning that L_t is not similar to a particular one in **WM** (when $\sigma < \mu$), then L_t is more likely to be a new location. The transition model $\mathbf{p}(S_t \mid S_{t-1} = i)$ is used to predict the distribution of S_t, given each state of the distribution S_{t-1} in accordance with the robot's motion between t and $t-1$. Combined with $\mathbf{p}(S_{t-1} = i \mid L^{t-1})$ (i.e., the recursive part of the filter), this constitutes the belief of the next loop closure. The transition model is expressed as in (**?**).

- $\mathbf{p}(S_t = -1 \mid S_{t-1} = -1) = 0.9$, the probability of a new location event at time t given that no loop closure occurred at time $t-1$.
- $\mathbf{p}(S_t = i \mid S_{t-1} = -1) = \frac{0.1}{N_{WM}}$ with $i \in [0, t_n]$, the probability of a loop closure event at time t given that no loop closure occurred at time $t-1$. N_{WM} is the number of locations in **WM** of the current iteration.
- $\mathbf{p}(S_t = -1 \mid S_{t-1} = j) = 0.1$ with $j \in [0, t_n]$, the probability of a new location event at time t given that a loop closure occurred at time $t-1$ with j.
- $\mathbf{p}(S_t = i \mid S_{t-1} = j)$ with $i, j \in [0, t_n]$, the probability of a loop closure event at time t given

that a loop closure occurred at time $t-1$ on a neighbor location. The probability is defined as a discrete Gaussian curve centered on j and where value are non-null for exactly eight neighbors ($for\ i = j-4,\ldots,j+4$) and their sum are 0.9.

2.1.4 Loop Closure Hypothesis Selection

When $\mathbf{p}(S_t \mid L^t)$ has been updated and normalized, highest hypothesis of $\mathbf{p}(S_t \mid L^t)$ greater than the loop closure threshold T_{loop} is selected. Because a loop closure is generally diffused to its neighbors (neighbor locations share some similar words between them), for each probability $\mathbf{p}(S_t = i \mid L_t)$ *where* $i \geq 0$, the probablilities of its neighbors in the range defined by the transition model $\mathbf{p}(S_t = i \mid S_{t-1} = j)$ are summed together. Note that for a selection to occur, aminimum of T_{minHyp} locations must be in **WM**: when the graph is initialized, the first locations added to **WM** have automatically a high loop closure probability (after normalization by the equation (2.2)), therefore wrong loop closures could be accepted if the hypothesis are over T_{loop}.

If there is loop closure hypothesis $\mathbf{p}(S_t = i \mid L^t)$ *where* $i \geq 0$ is over T_{loop}, the hypothesis $S_t = i$ is then accepted, L_t is merged with the old location L_i: the weight of L_t is increased by the one of L_i plus one and the neighbor links of L_i are added to L_t. After updatoing L_t, unlike **Rehearsal**, L_i is not deleted from **WM**. For the constancy of the Bayesian filter with the next acquired images, the associated loop closure hypothesis S_i must be evaluated until L_i is not anymore in the neighborhood of the highest loop closure hypothesis, after which it is removed.

2.1.5 Retrieval

Neighbors of location L_i in **LTM** are transferred back into **WM** if $\mathbf{p}(S_t = i \mid L^t)$ is the heighest probability. Because this step is time consuming, a maximum of two locations are retrived at each iteration (choosen inside the neighboring range defined in Section 2.1.3). The visual dictionary is updated with the words associated with the signatures of the retrieved locations. Common words from the retrieved signatures still exist in the dictionary, so a simple reference is added between these words and the corresponding signatures. For words that are not present anymore (because they were removed from the dictionary when the locations were transferred), they are matched to those in the dictionary to find if more recent words represent the same **SURF** features. This step is particularly important because the new words added from the new signature z_t may be identical to the previously transferred words. For matched words, the old words are replaced by the new ones in the retrieved signatures. All the references in the **LTM** are changed and the old words are permanently removed. If some words are still unmatched, they are simply added to the dictionary.

2.1.6 Transfer

When processing time becomes greater than T_{time}, the oldest location with the lowest weight is transferred from **WM** to **LTM**. T_{time} must be set to allow the robot to process the perceived

images in real-time. Higher T_{time} means that more locations (and implicitly more words) can be kept in the **WM**, and more loop closure hypotheses can be kept to better represent the overall environment. T_{time} must therefore be set according to the robot's CPU capabilities, computational load and operating environmetn. If T_{time} is set to higher than the image acquisition time, the algorithm intrinsically uses an image rate corresponding to T_{time}, with 100% CPU usage. As a rule of thumb, T_{time} can be set to about 200 to 400 ms smaller to the image acquisition rate at 1 Hz, to ensure that all images are processed under the image acquisition rate, even if the processing time goes over T_{time}. (T_{time} then corresponds to the average processing time of an image by the algorithm). So, for an image acquisition rate of 1 Hz, T_{time} can be set to between 600 and 800 ms.

Because the most expensive step of the algorithm is to rebuild the visual dictionary, process time can be regulated by changing the dictionary size, which indirectly influences the **WM** size. A signature of a location transferred to **LTM** removes its word references from the visual dictionary. If a word does not have reference to a signature anymore, it is transferred into **LTM**. While the number of words transferred from the dictionary is less than the number of words added from the new or retrieved locations, more locations are transferred. At the end of this process, the dictionary size is smaller than before the new words from the new and retrieved locations were added, thus reducing the time required to update the dictionary for the next acquired image. Saving the transferred locations into the database is done asynchronously using a background thread, leading to a minimal time overhead. Note also that to be able to evaluate appropriately loop closure hypotheses using the discrete Bayesian filter, the retrieved locations of the current iteration are not allowed to be transferred.

2.2 ROS2 Working Framework

The ROS2 (Robot Operating System 2) working framework is another theoretical foundation of this engineering project. ROS2 is an open-source robot operating system that provides a flexible framework for writing robot software. The ROS2 working framework includes some core concepts and components that work together to assist developers in building feature-rich robotic applications. To help readers better understand this project, a brief introduction to some core concepts and components of ROS2 is provided here. For more information, please refer to the relevant literature (**?**).

2.2.1 ROS graph

The ROS graph is a network of ROS 2 elements processing data together at the same time. It encompasses all executables and the connections between them if you were to map them all out and visualize them. Appendix A provides ROS2 graphs from different stages of this engineering project, showing how the ROS2 graph evolves as the engineering progresses.

2.2.2 Node

Each node in ROS should be responsible for a single, modular purpose, e.g. controlling the wheel motors or publishing the sensor data from a laser range-finder. As shown in Figure 2.2, each node can send and receive data from other nodes via topics, services, actions, or parameters.

A full robotic system is comprised of many nodes working in concert. In ROS 2, a single executable (C++ program, Python program, etc.) can contain one or more nodes. A node is a fundamental ROS 2 element that serves a single, modular purpose in a robotics system.

2.2.3 Topic

ROS 2 breaks complex systems down into many modular nodes. Topics are a vital element of the ROS graph that act as a bus for nodes to exchange messages. As shown in Figure 2.3, nodes can publish messages to a topic, and other nodes can subscribe to that topic to receive those messages. This allows for a flexible and decoupled communication mechanism between nodes. A node may publish data to any number of topics and simultaneously have subscriptions to any number of topics. Topics are one of the main ways in which data is moved between nodes and therefore between different parts of the system.

2.2.4 Service

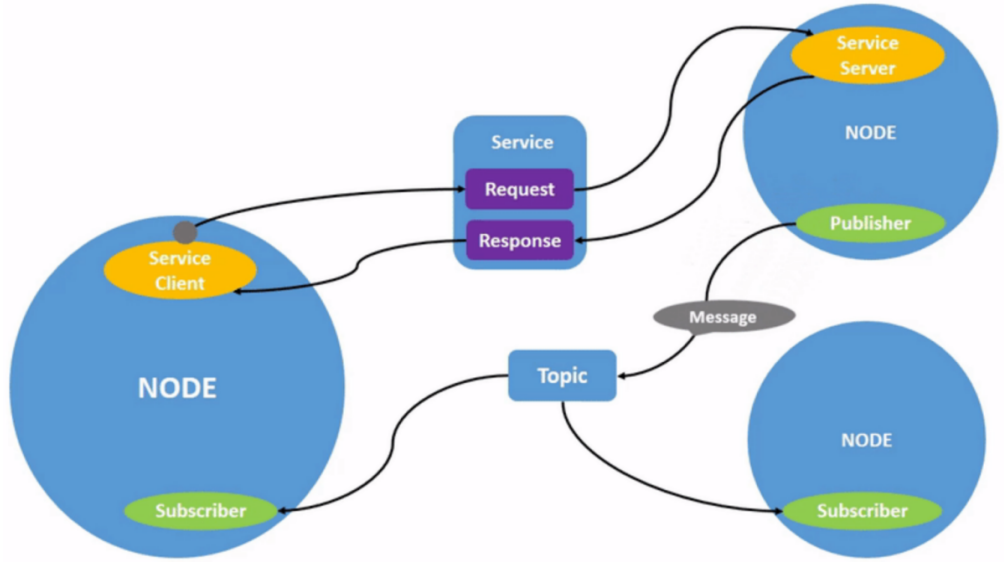
Services are another method of communication for nodes in the ROS graph. Services are based on a call-and-response model versus the publisher-subscriber model of topics. While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client. Nodes can communicate using services in ROS 2. Unlike a topic - a one way communication pattern where a node publishes information that can be consumed by one or more subscribers - a service is a request/response pattern where a client makes a request to a node providing the service and the service processes the request and generates a response.

2.2.5 Action

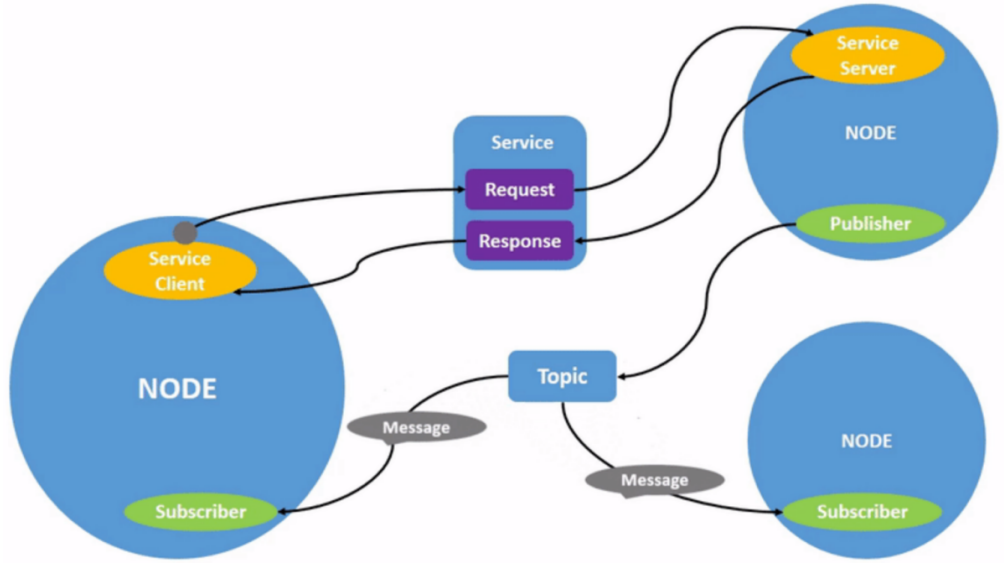
Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and a result.

As shown in Figure 2.5, actions are built on topics and services. Their functionality is similar to services, except actions can be canceled. They also provide steady feedback, as opposed to services which return a single response.

Actions use a client-server model, similar to the publisher-subscriber model (described in the section 2.2.3). An “action client” node sends a goal to an “action server” node that acknowledges the goal and returns a stream of feedback and a result.

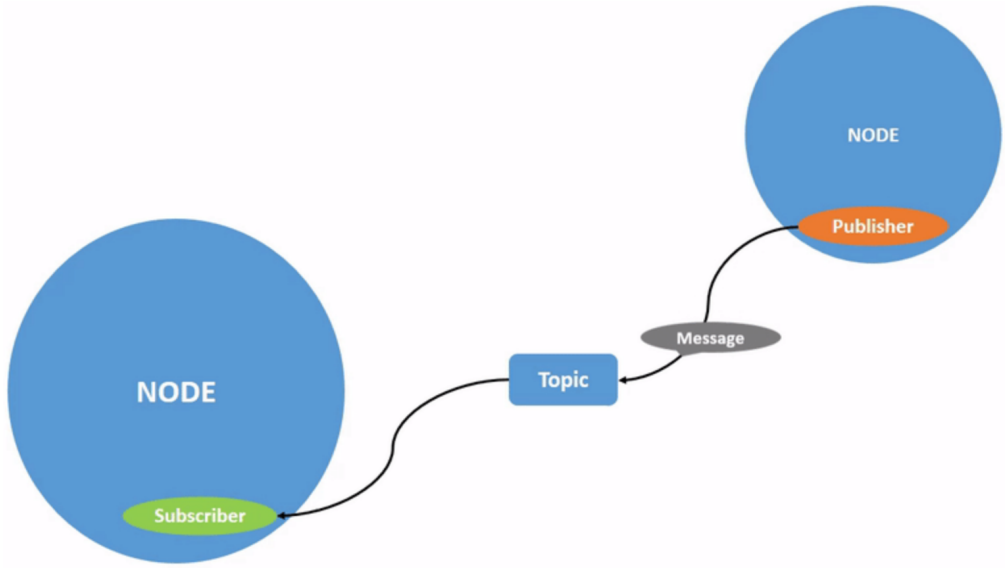


(a) Publish message

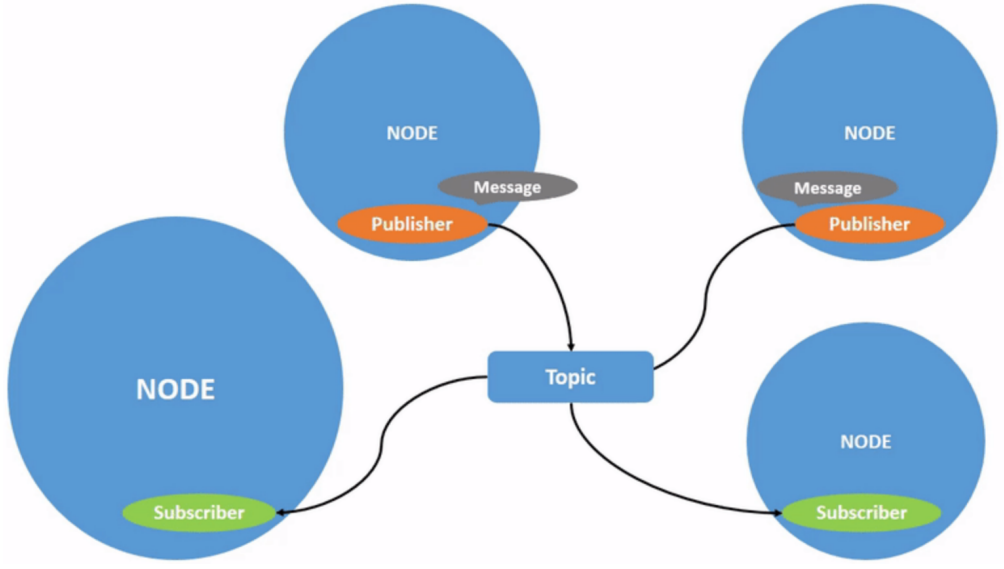


(b) Subscribe messages

Figure 2.2: ROS2 Nodes (Publisher) & (Subscribers)

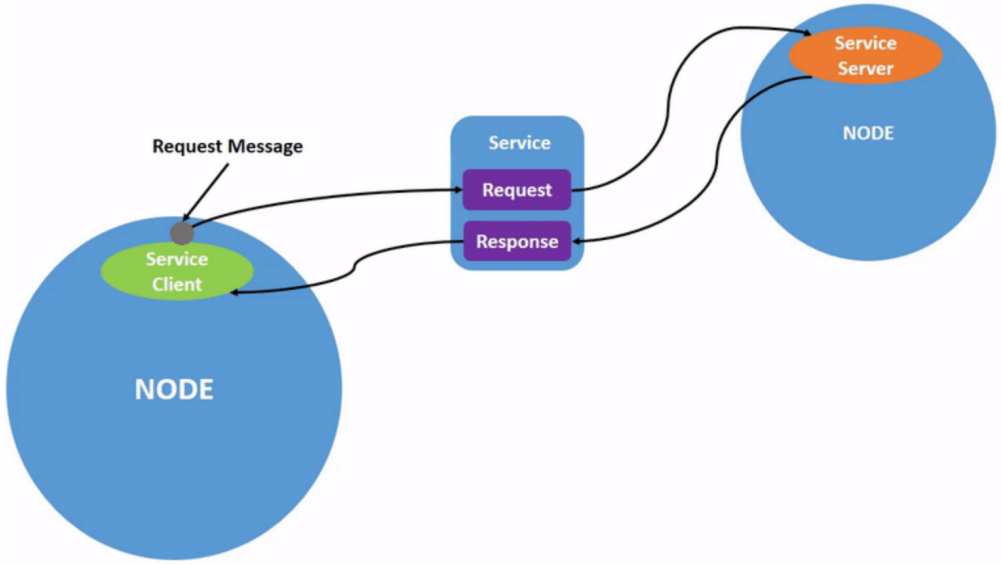


(a) one-to-one

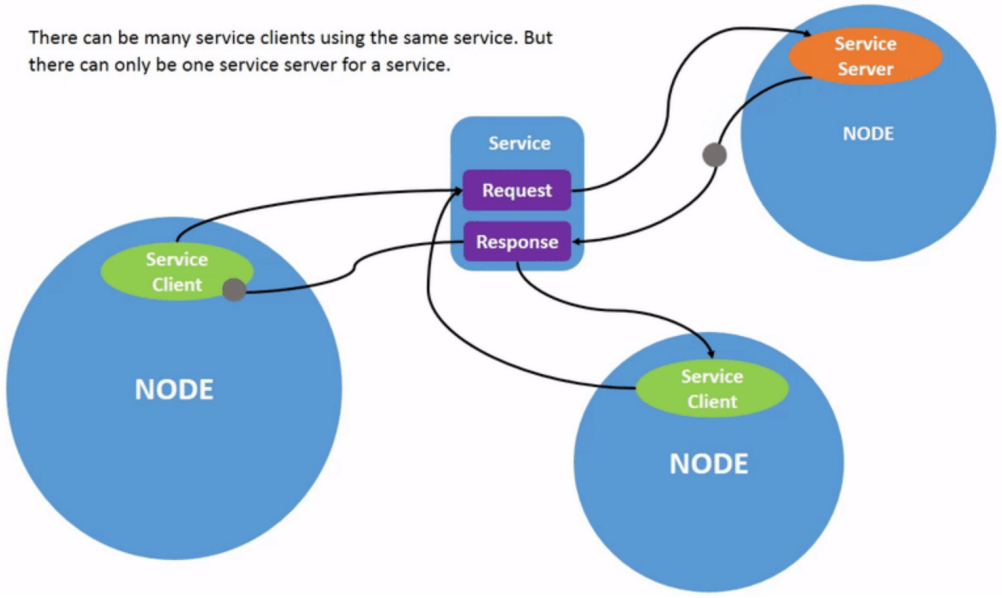


(b) many to many

Figure 2.3: ROS2 Topics (one-to-one) & (many to many)



(a) one-to-one



(b) many to one

Figure 2.4: ROS2 Services (one-to-one) & (many to one)

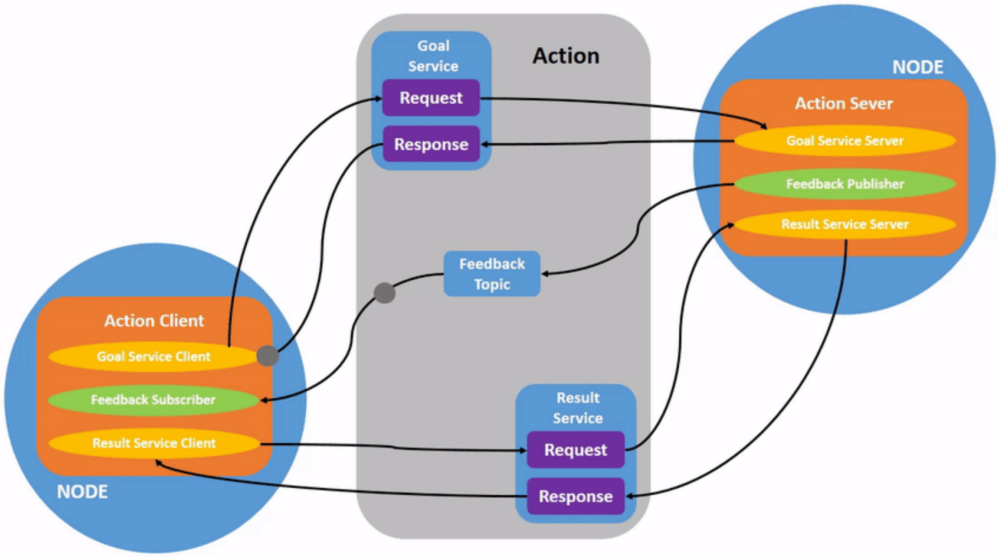


Figure 2.5: ROS2 Action

A robot system would likely use actions for navigation. An action goal could tell a robot to travel to a position. While the robot navigates to the position, it can send updates along the way (i.e. feedback), and then a final result message once it's reached its destination.

2.2.6 Parameter

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings, and lists. In ROS 2, each node maintains its own parameters.

Nodes have parameters to define their default configuration values. You can get and set parameter values from the command line. You can also save the parameter settings to a file to reload them in a future session.

2.2.7 TF2

tf2 is the second generation of the transform library, which lets the user keep track of multiple coordinate frames over time. tf2 maintains the relationship between coordinate frames in a tree structure buffered in time, and lets the user transform points, vectors, etc between any two coordinate frames at any desired point in time.

Figure 2.6 shows the tf2 tree structure used in this engineering project. The root frame is the `map` frame, which is the global coordinate frame of the environment. The `odom` frame is the odometry frame, which is used to track the robot's position and orientation over time. The `zed_camera_link` frame is the robot's base frame, which is used to represent the robot's position and orientation in the environment. The `zed_left_camera_frame` and

`zed_right_camera_frame` frames are the left and right camera frames, respectively, which are used to represent the stereo camera's position and orientation.

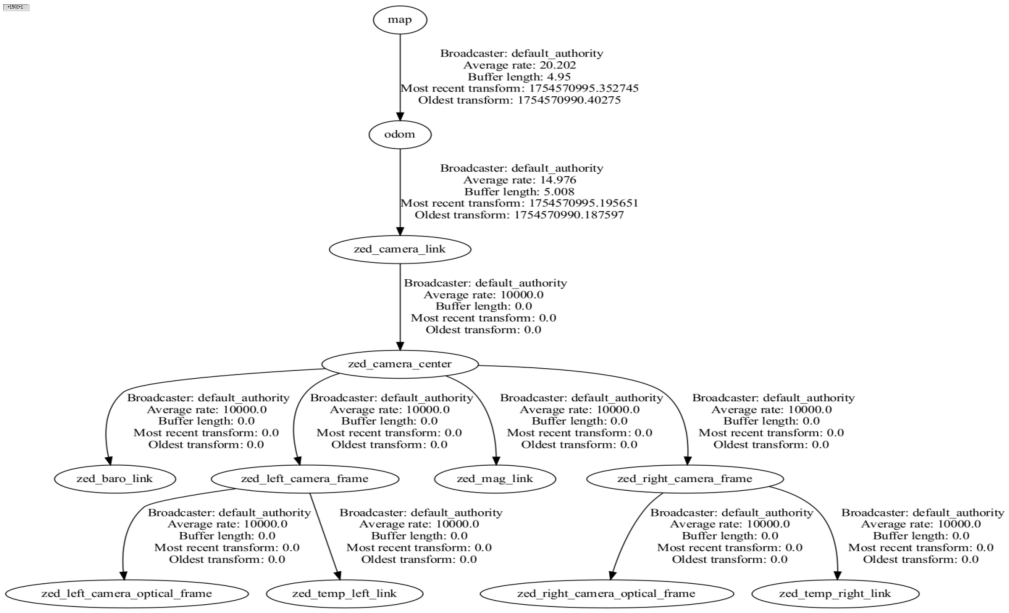


Figure 2.6: TF2 tree structure used in this engineering project.

Chapter 3

Engineering Design

This chapter will provide a description of the system design for the engineering, including use case, architecture and data flow. Next, we will introduce them one by one.

3.1 Use Case

First of all, we will introduce the use case of the engineering. The use case is shown in Figure 3.1. There are four actors in the use case:

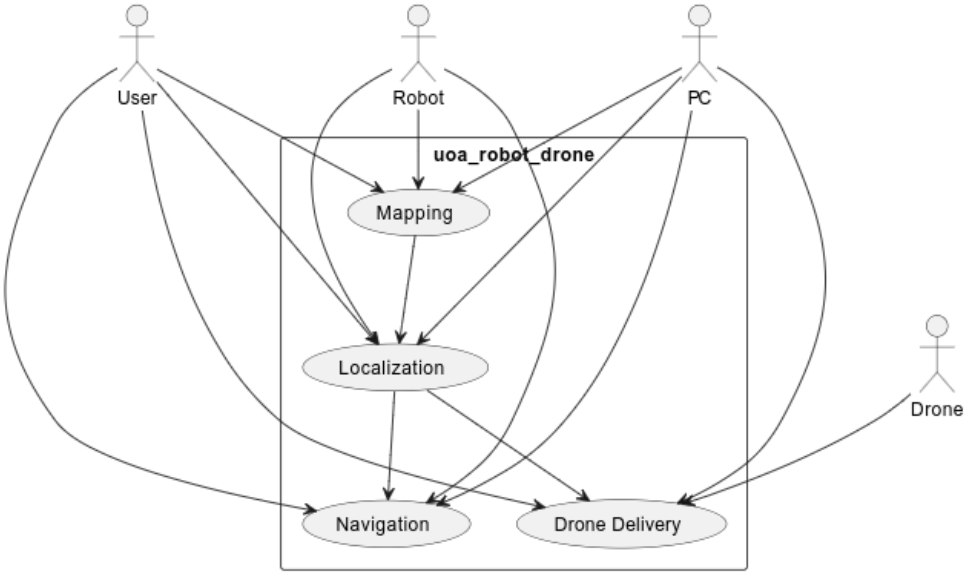


Figure 3.1: Use Case

- **User**: The user can control the robot to move around, set a target position, and monitor the robot's status.
- **Robot**: The robot can perform mapping, localization, and navigation tasks, and can also

deliver items using a drone.

- **Drone**: The drone can be controlled by the robot to deliver items to a specified target position.
- **PC**: The PC is a personal computer that runs the uoa_robot_drone engineering package, which includes nodes for monitoring the robot's status and controlling the drone.

There are four main use cases in the use case diagram:

- **Mapping**: The robot can perform mapping tasks to create a map of the environment.
- **Localization**: The robot can determine its position in the environment using the map created during the mapping stage.
- **Drone Delivery**: The robot can control the drone to deliver items to a specified target position.
- **Navigation**: The robot can navigate to a specified target position using the map created during the mapping stage and the localization information.

The following section is Engineering System Architecture.

3.2 Engineering System Architecture

This distributed system architecture consists of three main components: Raspberry Pi, Jetson Orin Nano, and Personal Computer. Each component integrates multiple hardware and software modules, as shown in Figure 3.2, to collaboratively accomplish mapping, localization, navigation, and drone delivery tasks.

- **Raspberry Pi**:
  - **Turtlebot3**:
    - `turtlebot3_bringup`: Launches core robot nodes and drivers.
    - `turtlebot3-msgs`: Defines message types for robot communication.
  - `Dynamixel SDK`: Motor control library for actuators.
  - `python3-colcon`: ROS2 build and package management tool.
  - `coin_d4_driver`: Driver for additional hardware components.
  - `OpenCR`: Controller board for sensors and actuators.
  - `ROS2 Humble Hawksbill`: Robot Operating System for distributed communication.

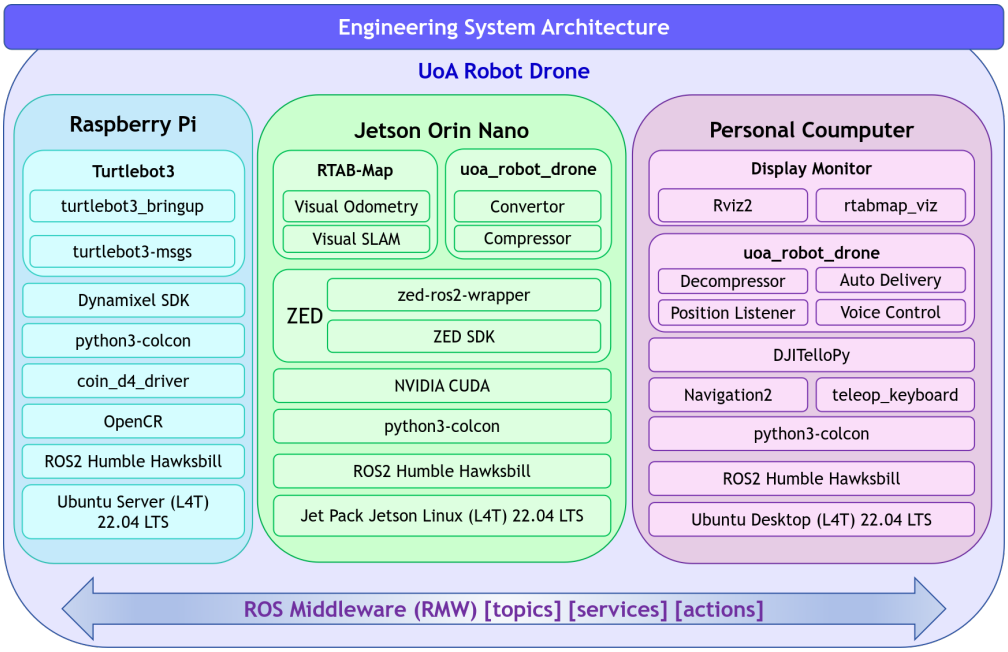


Figure 3.2: Engineering System Architecture

  - `Ubuntu Server (L4T) 22.04 LTS`: Operating system for the Raspberry Pi.

- **Jetson Orin Nano**:
  - **RTAB-Map**:
    - Visual Odometry: Estimates robot motion from camera data.
    - Visual SLAM: Simultaneous localization and mapping.
  - **uoa_robot_drone**:
    - Convertor: Image format conversion ($bgra8 \rightarrow bgr8$) for efficient processing.
    - Compressor: Compresses image data for transmission.
  - **ZED**:
    - `zed-ros2-wrapper`: ROS2 interface for ZED stereo camera.
    - `ZED SDK`: Provides stereo vision and depth data.
  - `NVIDIA CUDA`: GPU acceleration for vision algorithms.
  - `python3-colcon`, `ROS2 Humble Hawksbill`: As above.
  - `Jet Pack Jetson Linux (L4T) 22.04 LTS`: Operating system for Jetson Orin Nano.

- **Personal Computer**:
  - **Display Monitor**:
    - `Rviz2`: Visualization of robot state and environment.
    - `rtabmap_viz`: SLAM and mapping visualization.
  - **uoa_robot_drone**:
    - Decompressor: Decompresses image data from Jetson.
    - Auto Delivery: Controls drone delivery tasks.
    - Position Listener: Monitors robot/drone position.
    - Voice Control: Enables voice command interface.
  - `DJITelloPy`: Python library for controlling DJI Tello drone.
  - `Navigation2`: Path planning and autonomous navigation.
  - `teleop_keyboard`: Manual robot control via keyboard.
  - `python3-colcon`, `ROS2 Humble Hawksbill`: As above.
  - `Ubuntu Desktop (L4T) 22.04 LTS`: Operating system for the PC.

All modules communicate via ROS2 middleware (topics, services, actions), enabling seamless data exchange and coordinated operation across the distributed system.

The next section will introduce the **Data Flow** of the engineering system.

3.3 Data Flow

The data flow is shown in Figure 3.3.

Figure 3.3 Legend explanation:

- Ellipse: represents work packages or nodes (teleop_keyboard, rtabmap_viz, rviz2 represent nodes, others are work packages)
- Nested rounded Rectangle:
  - Outer layer represents work package (e.g., uoa_robot_drone)
  - Inner layer represents the device that the package is deployed
- Rectangular shape:
  - Topics starting with "/" represent ROS topics
  - Topics starting with letters represent data

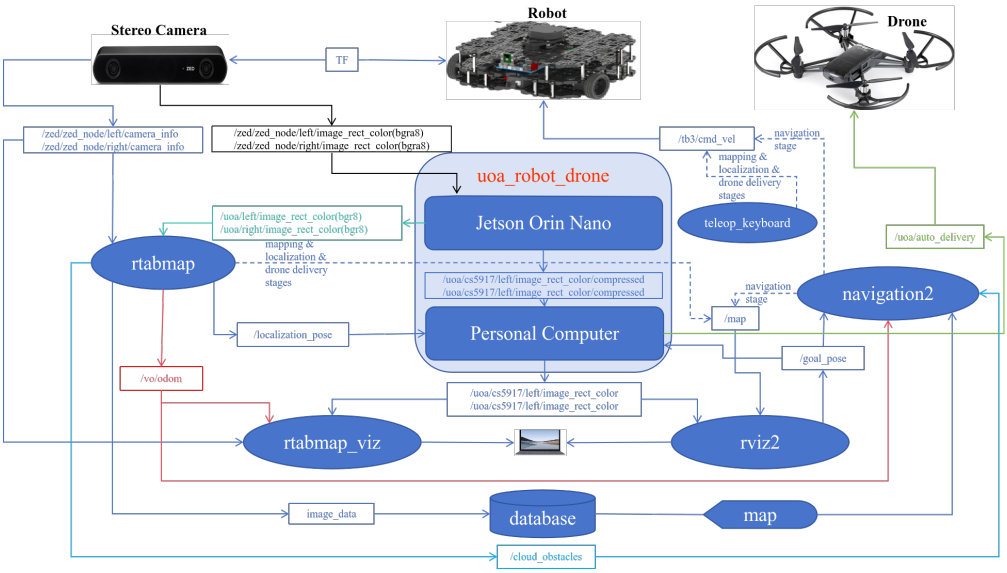


Figure 3.3: Data Flow

- Arrow: Indicates data flow direction, where:
  - Solid arrow indicates a continuous data flow
  - Dashed arrow indicates a data flow that exists only in certain stages, such as the /map topic in the rtabmap package existing during the mapping stage but not during navigation, while the /map topic published by the navigation2 package's map_server node exists during navigation but not during mapping.
- Cylinder: Represents data storage
- Rounded triangle: Represents map data
- TF: Represents coordinate transformation, which is used to transform the coordinate system of the data and more details seen section 2.2.7.

Next, we will introduce the data flow in detail.

3.3.1 Stereo Camera Captures Left and Right Camera Image Data

After the camera captures external image data, it publishes two types ("Camera Info" and "Image") of four topics of image data through the `/zed/zed_node` node:

- **Camera Info topics:** Publishes camera intrinsic and extrinsic parameters, including focal length, distortion coefficients, etc. The two published topic names are:

  - /zed/zed_node/left/camera_info
  - /zed/zed_node/right/camera_info

  The camera_info topics can be directly passed down to the rtabmap node for use.

- **Image topics:** Publishes rectified left and right image data, including information such as image width, height, and encoding format. The two published topic names are:
  - /zed/zed_node/left/image_rect_color
  - /zed/zed_node/right/image_rect_color

The two rectified image topics are in bgra8 format. To reduce network transmission data volume and improve computational efficiency, the data is first preprocessed by the `convert_image_format` node in the `uoa_robot_drone` package deployed on the Jetson Orin Nano.

3.3.2 Image Preprocessing in `uoa_robot_drone` Package on Orin

The `convert_image_format` node in the `uoa_robot_drone` package deployed on the Jetson Orin Nano subscribes to the two rectified image topics published by the `/zed/zed_node` node to obtain the left and right camera bgra8 format image data, converts them to bgr8 format, and publishes them to the `/uoa/left/image_rect_color` and `/uoa/right/image_rect_color` topics. These topics are then subscribed to by two republish nodes (`orin_repub_left` and `orin_repub_right`) in the `uoa_robot_drone` package and the `stereo_sync` node in the `rtabmap` package (specifically named `rtabmap_sync`). To reduce network transmission data volume, the two republish nodes compress the subscribed data from `/uoa/left/image_rect_color` and `/uoa/right/image_rect_color` into compressed format and publish them to `/uoa/cs5917/left/image_rect_color/compressed` and `/uoa/cs5917/right/image_rect_color/compressed` topics, which are then subscribed to by two uncompressed nodes (`uoa_pc_repub_left` and `uoa_pc_repub_right`) and uncompressed and finally are used by monitoring nodes (`rtabmap_viz` and `rviz2`) deployed on the PC. Therefore, the nodes in the `uoa_robot_drone` package deployed on the Jetson Orin Nano will publish the following four topics:

- `/uoa/orin/left/image_rect_color`
- `/uoa/orin/right/image_rect_color`
- `/uoa/cs5917/left/image_rect_color/compressed`
- `/uoa/cs5917/right/image_rect_color/compressed`

3.3.3 rtabmap Package for Mapping and Localization

Next, the nodes in the rtabmap package will subscribe to the data from the following four topics:

- `/uoa/orin/left/image_rect_color`
- `/uoa/orin/right/image_rect_color`
- `/zed/zed_node/left/camera_info`
- `/zed/zed_node/right/camera_info`

to perform calculations and publish relevant topics based on different stages (mapping or localization), mainly including:

- `/vo/odom`
  Visual odometry data
- `/localization_pose`
  Current robot pose data
- `/cloud_obstacles`
  Obstacle point cloud data
- `/map`
  Publishes the map data generated during the mapping process. As shown in the figure, the connection line to this topic from `rtabmap` is a dashed line, indicating that this topic is published during the mapping stage but not during navigation. During navigation, the `/map` topic is published by the `map_server` node in the `navigation2` package.

In fact, here we only introduce some important topics published by the nodes in the rtabmap package. Other topics can be found in Appendix A.

3.3.4 `uoa_robot_drone` Package on PC Receives Image, Robot Current Pose, and Target Pose Data, and Publishes Control Commands to Drone

Next, the nodes in the `uoa_robot_drone` package deployed on the PC will subscribe to the data from the following topics:

- `/uoa/cs5917/left/image_rect_color/compressed`
- `/uoa/cs5917/right/image_rect_color/compressed`
- `/localization_pose`

- `/goal_pose`

Then, it will perform the following tasks:

- Decompress the data from `/uoa/cs5917/left/image_rect_color/compressed` and `/uoa/cs5917/right/image_rect_color/compressed` topics to obtain the left and right camera bgr8 format image data.
- Publish the left and right camera bgr8 format image data to the following topics:
  - `/uoa/cs5917/left/image_rect_color`
  - `/uoa/cs5917/right/image_rect_color`
- When in the drone delivery stage, calculate the distance and angle to the target position based on the current robot pose data obtained from the `/localization_pose` topic and the target pose data obtained from the `/goal_pose` topic. If the distance is less than 0.1 meters and the angle is less than 0.1 radians for three consecutive calculations, a control command with delivery_signal=true is published to the `/uoa/auto_delivery` topic, indicating that the drone can start the delivery task.

3.3.5 `rtabmap_viz` Node Displays Loop Closure Detection and Odometry Image Data

The `rtabmap_viz` node will subscribe to the data from the following topics:

- `/uoa/cs5917/left/image_rect_color`
- `/uoa/cs5917/right/image_rect_color`
- `/zed/zed_node/left/camera_info`
- `/zed/zed_node/right/camera_info`

Then, it will display the Odometry and loop closure images on the screen.

3.3.6 rviz2 Node Displays Map Point Cloud Data and Robot Pose

The `rviz2` node will subscribe to the data from the following topics:

- `/map` Receives map data through this topic
- `/uoa/cs5917/left/image_rect_color` Receives left camera image data through this topic

- `/uoa/cs5917/right/image_rect_color` Receives right camera image data through this topic

Then, it will display the map data and the video data seen by the robot in its current field of view on the screen. In `rviz2`'s map, after clicking the "2D Goal Pose" button, move the mouse to the map and click the left mouse button to set the target point. `rviz2` will publish the pose data of this target point to the `/goal_pose` topic, which will be subscribed to by the `uoa_robot_drone` and related nodes in the `navigation2` package to complete subsequent drone delivery tasks and navigation tasks.

3.3.7 `navigation2` Package for Navigation

The nodes in the `navigation2` package will subscribe to the data from the following topics:

- `/goal_pose` Target pose data
- `/vo/odom` Visual odometry data
- `/cloud_obstacles` Obstacle point cloud data
- `map` Map data from the database

Then, it will perform the following tasks:

- Based on the target pose data obtained from the `/goal_pose` topic, the visual odometry data obtained from the `/vo/odom` topic, and the map data, it will plan the global path from the robot's current pose to the target pose and the path within the field of view.
- Plan the action sequence for the robot to execute based on the path within the field of view and the global path.
- Publish the action sequence to the `/tb3/cmd_vel` topic for the `Robot` to subscribe and execute the action sequence to move towards the target position.
- When the robot detects obstacles in its path through the obstacle point cloud data published on the `/cloud_obstacles` topic, it will replan the path to avoid the obstacles.

3.3.8 `teleop_keyboard` Node for Manual Robot Control

During the mapping, localization, and drone delivery stages, user can manually control the `Robot` to move forward, backward, turn left, and turn right by publishing control commands to the `/tb3/cmd_vel` topic using the keyboard through the `teleop_keyboard` node.

Chapter 4

Implementation

To enable readers to fully reproduce this engineering, this chapter will detail the implementation process, including hardware selection, environment setup, and software development.

4.1 Hardware Selection

The hardware devices used in this engineering project are Turtlebot3 Waffle Pi, ZED 2 stereo camera, Jetson Orin Nano, INIU 65W 20000mAh power bank, DJI Tello EDU drone, and Lenovo laptop with 16GB RAM.

4.1.1 Robot Platform – Turtlebot3 Waffle Pi

Turtlebot3 Waffle Pi is an open-source robot platform based on Raspberry Pi SBC, suitable for chassis. Its dimension graph is shown in Figure 4.1, and its components are shown in Figure 4.2. For more detailed specifications and component lists of Turtlebot3 Waffle Pi, please refer to Appendix **??**.

4.1.2 Stereo Vision Camera – ZED 2

ZED 2 is a high-performance stereo vision camera, used for visual SLAM and environment perception. Its front view is shown in Figure 4.3. For more detailed specifications of ZED 2, please refer to Appendix **??**.

4.1.3 Edge Computing Unit – Jetson Orin Nano

Regarding the selection of the edge computing unit, NVIDIA has released various configurations of singleboard computers, including 4GB, 8GB, 16GB, 32GB, 64GB, and 128GB. Considering the actual requirements of this project, we chose the Jetson Orin Nano 8GB version. It is a lightweight embedded edge computing singleboard computer launched by NVIDIA, suitable for AI inference and edge computing tasks. Its layout diagram is shown in Figure 4.4, and its components are listed in Table 4.1.

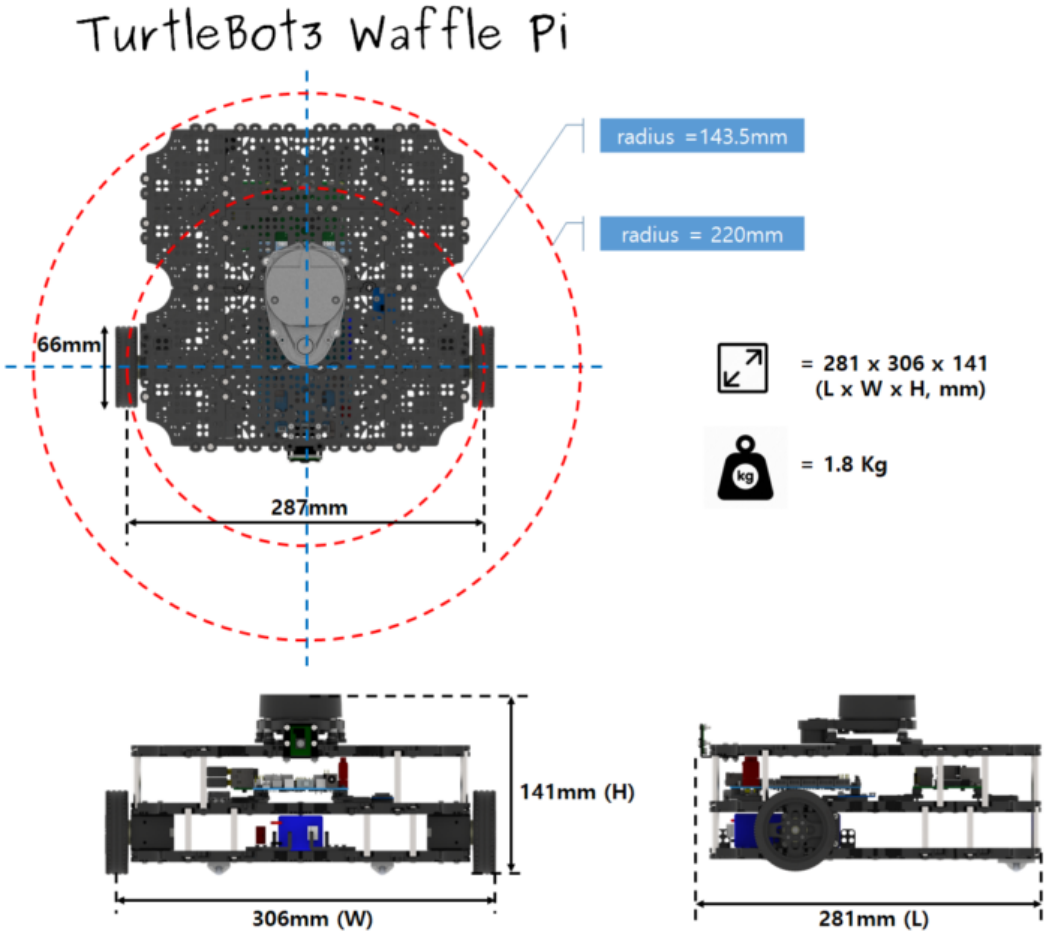


Figure 4.1: Turtlebot3 Waffle Pi Dimensions and Mass

4.1.4 Power Supply – INIU Power Bank

When selecting a power bank to supply power to the Jetson Orin Nano, we considered stability and chose a product with a voltage of up to 20V and a maximum output power of at least 65W. The INIU Power Bank was selected for this purpose. It has a capacity of 20000mAh and can output 65W at 20V, which is sufficient to power the Jetson Orin Nano. Its appearance is shown in Figure 4.5. For more detailed specifications of INIU Power Bank, please refer to Appendix **??**.

4.1.5 Drone – DJI Tello EDU

To validate the feasibility of multiagent collaboration, this engineering selected the DJI Tello EDU drone. This drone is compact and lightweight, suitable for indoor flight and programming education. Its appearance is shown in Figure 4.6.

For more detailed specifications of DJI Tello EDU, please refer to Appendix **??**.

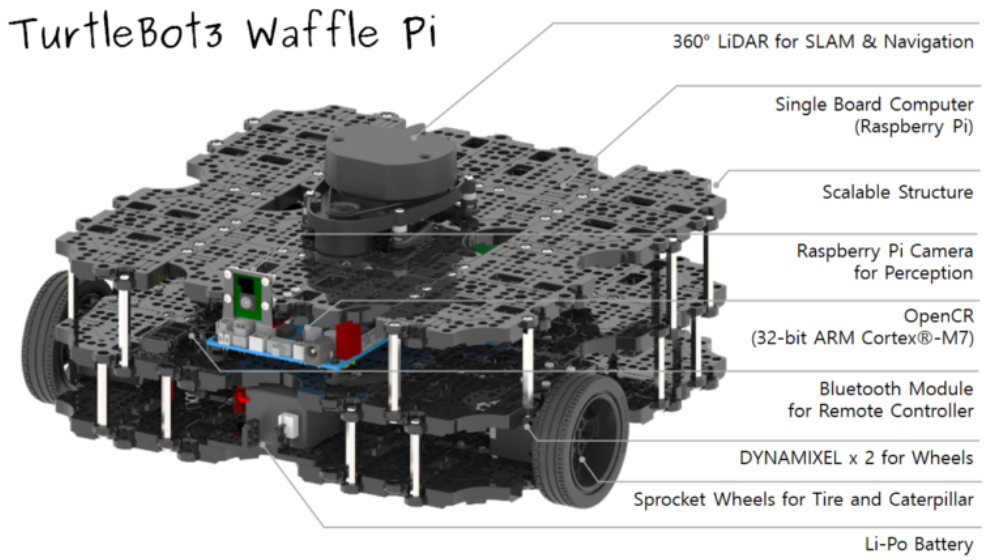


Figure 4.2: Turtlebot3 Waffle Pi Components (Note: The LiDAR sensor shown in this figure is not used in this project, as we employ pure vision-based SLAM)

Figure 4.3: ZED 2 Stereo Vision Camera

4.1.6 Monitoring System – Lenovo Laptop Yoga 14s 2021

The monitoring system for this project uses a Lenovo laptop with 16GB of RAM, suitable for data processing and visualization tasks. For more detailed specifications of Lenovo Laptop Yoga 14s 2021, please refer to Appendix **??**.

4.2 Environment Setup

In this section, we will introduce how to set up the environment required for this engineering project, including the PC, Turtlebot3, and Jetson Orin Nano.

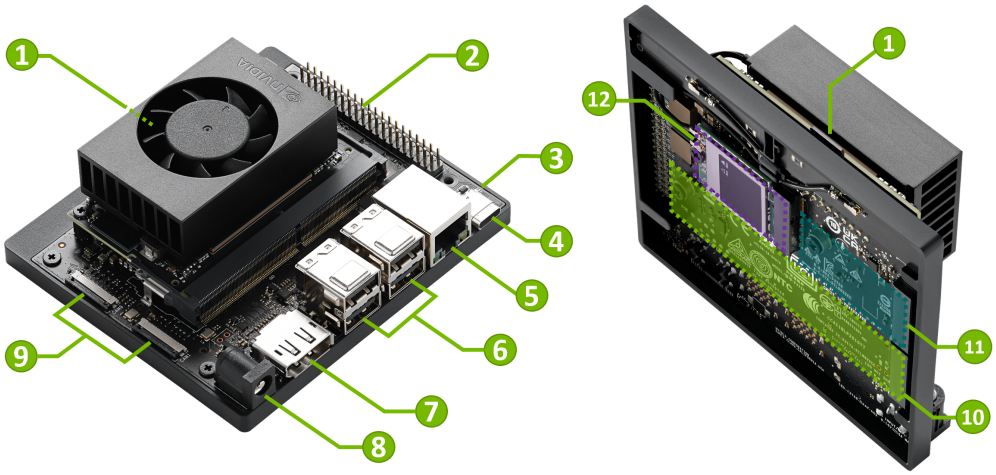


Figure 4.4: Jetson Orin Nano Layout Diagram

4.2.1 PC Setup

This engineering project uses a standard laptop as the PC. The specific hardware configuration is shown in Table **??**. On the PC, we mainly install the operating system, ROS 2, rtabmap package, the engineering package `uoa_robot_drone` and the development environment VS Code, and perform the corresponding environment configuration. Below, we will detail these steps.

Installing the Operating System

This engineering project uses Ubuntu 22.04 LTS as the operating system. *Note: To avoid issues, it is essential to install the OS using the **physical machine direct installation** method, not in a virtual machine (as Ubuntu on a virtual machine often fails during the Jetson Orin Nano flashing process). The installation steps are as follows:

1. Click on the link https://releases.ubuntu.com/22.04/ to download the ISO image file of Ubuntu 22.04 LTS, as shown in Figure 4.7.

2. Follow the tutorial Install Ubuntu Desktop to complete the OS installation, as shown in Figure 4.8.

Installing ROS 2

This engineering project uses ROS 2 Humble Hawksbill as the version. Follow the tutorial Ubuntu (deb packages) to install step by step, and execute the steps shown in Figure 4.9.

Installing Packages

On the PC, the main packages used in this engineering project are Turtlebot3, rtabmap, and navigation2. [On PC] The relevant installation commands are as follows:

Figure 4.5: INIU 65W 20000mAh Power Bank

Figure 4.6: DJI Tello EDU Drone

Table 4.1: Jetson Orin Nano Components

Mark	Name	Note
1	microSD card slot	
2	40-pin Expansion Header	
3	Power Indicator LED	
4	USB-C port	For data only
5	Gigabit Ethernet Port	
6	USB 3.2 Type-A ports (x4)	10Gbps
7	DisplayPort Output Connector	
8	DC Power Jack	5.5mm x 2.5mm
9	MIPI CSI Camera Connectors (x2)	22pin, 0.5mm pitch
10	M.2 Slot (Key-M, Type 2280)	PCIe 3.0 x4
11	M.2 Slot (Key-M, Type 2230)	PCIe 3.0 x2
12	M.2 Slot (Key-E, Type 2230)	(populated)

```
$ source /opt/ros/humble/setup.bash
$ mkdir -p ~/ros2_ws/src
$ cd ~/ros2_ws/src/
$ git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
$ git clone https://github.com/introlab/rtabmap.git
$ git clone -b ros2 https://github.com/introlab/rtabmap_ros.git
$ git clone -b humble https://github.com/ros-navigation/navigation2.git
$ git clone -b humble https://github.com/Jian-UOA/CS5917.git
$ sudo apt install python3-colcon-common-extensions
$ cd ~/ros2_ws
$ colcon build --symlink-install
```

Installing the Development Environment

This engineering project uses Visual Studio Code as the development environment. The installation steps are as follows:

1. Click on the link https://code.visualstudio.com/Download to download the appropriate version of Visual Studio Code (in this engineering project, the x64 version is selected), as shown in Figure 4.10.

2. Go to the directory where the downloaded file is saved, find the downloaded installation file (`code_1.103.1-1755017277_amd64.deb`), and follow the tutorial Install VS Code

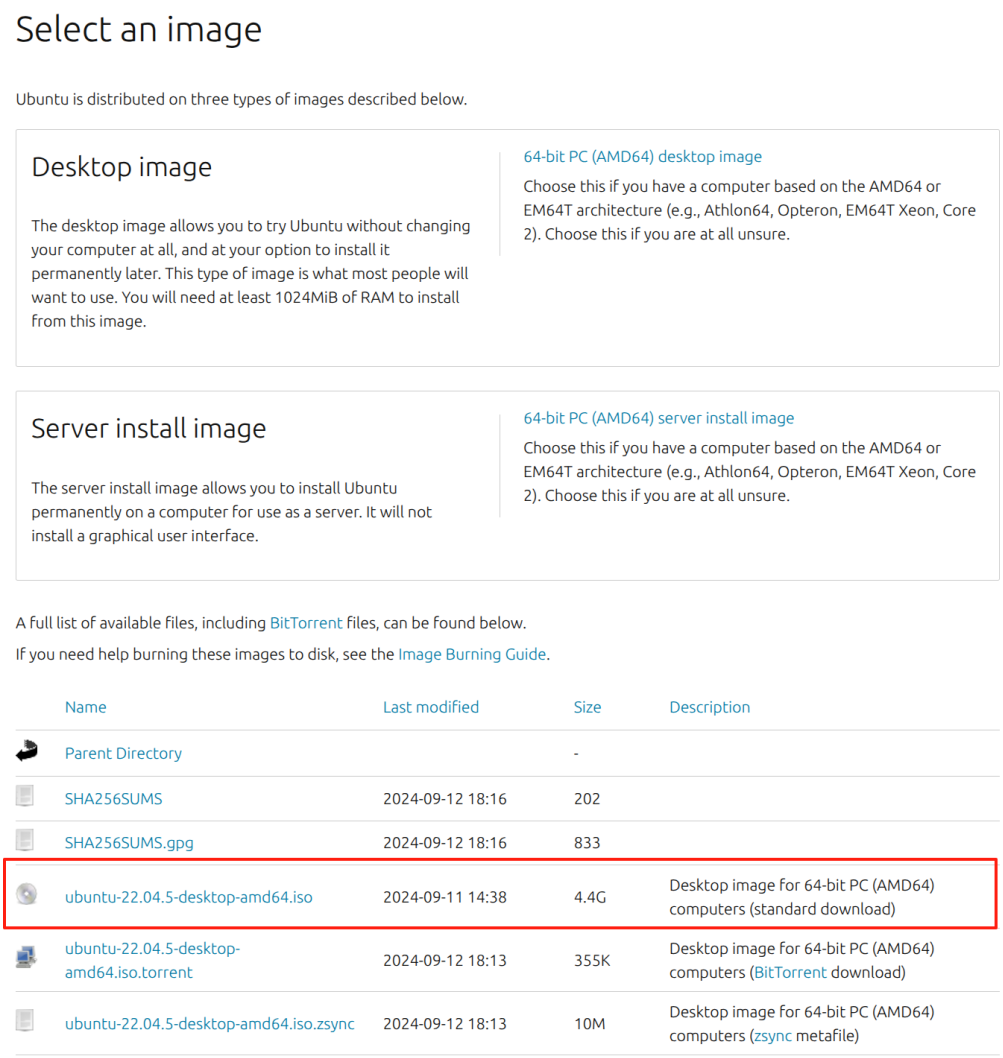


Figure 4.7: Downloading the ISO Image File of Ubuntu 22.04 LTS

on Linux to complete the installation, as shown in Figure 4.11.

3. Open Visual Studio Code, follow the example in the tutorial Tutorial: Get started with Visual Studio Code, complete the basic configuration, and open the workspace in the directory `/ros2_ws`. You should see content similar to Figure 4.12.

Environment Configuration

[On PC] The relevant configuration commands are as follows:

```
$ echo 'source␣~/ros2_ws/install/setup.bash' >> ~/.bashrc
$ echo 'source␣/opt/ros/humble/setup.bash' >> ~/.bashrc
```

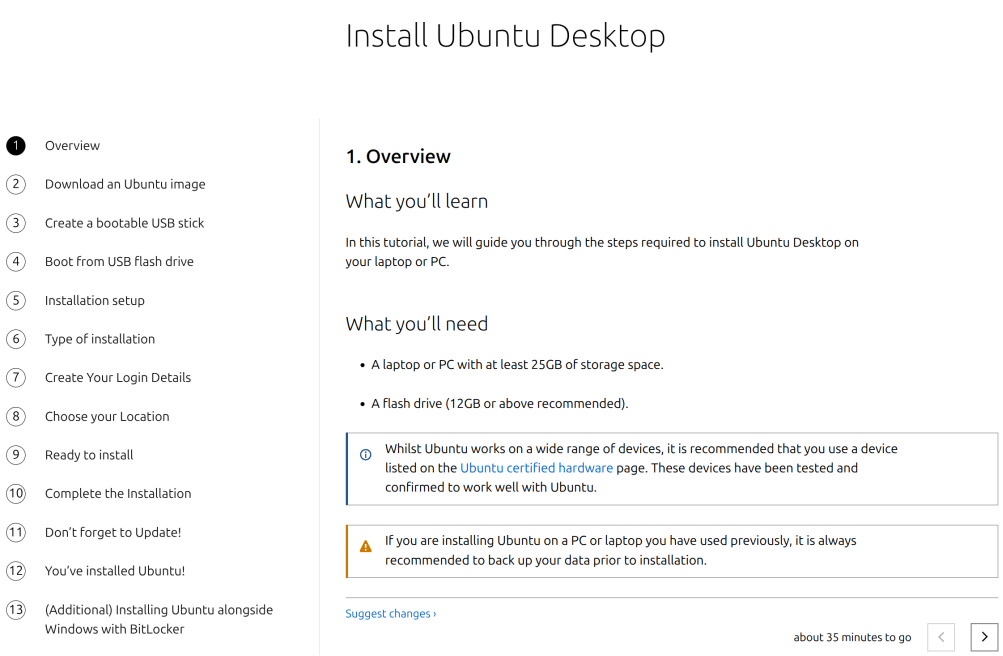


Figure 4.8: Installing Ubuntu 22.04 LTS

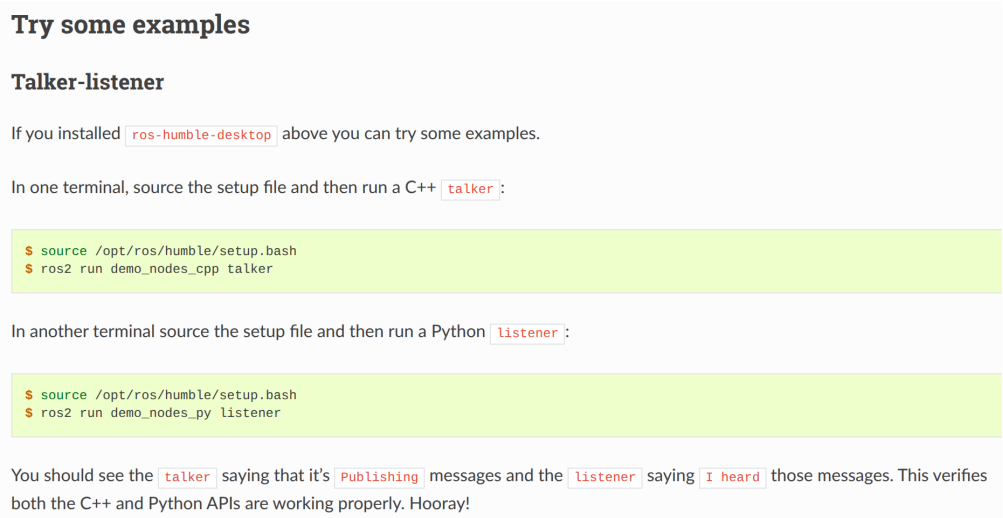


Figure 4.9: Installing ROS 2 Humble Hawksbill

```
$ echo "export␣TURTLEBOT3_MODEL=waffle_pi" >> ~/.bashrc
$ echo 'export␣ROS_DOMAIN_ID=30␣#TURTLEBOT3' >> ~/.bashrc
$ echo 'export␣RMW_IMPLEMENTATION=rmw_fastrtps_cpp' >> ~/.bashrc
$ source ~/.bashrc
```

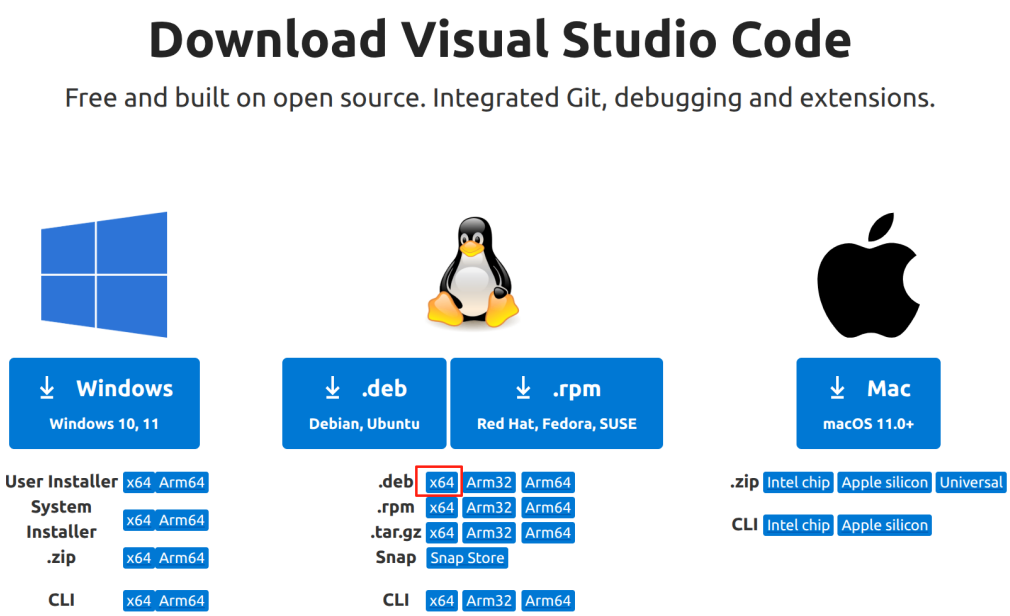


Figure 4.10: Downloading Visual Studio Code

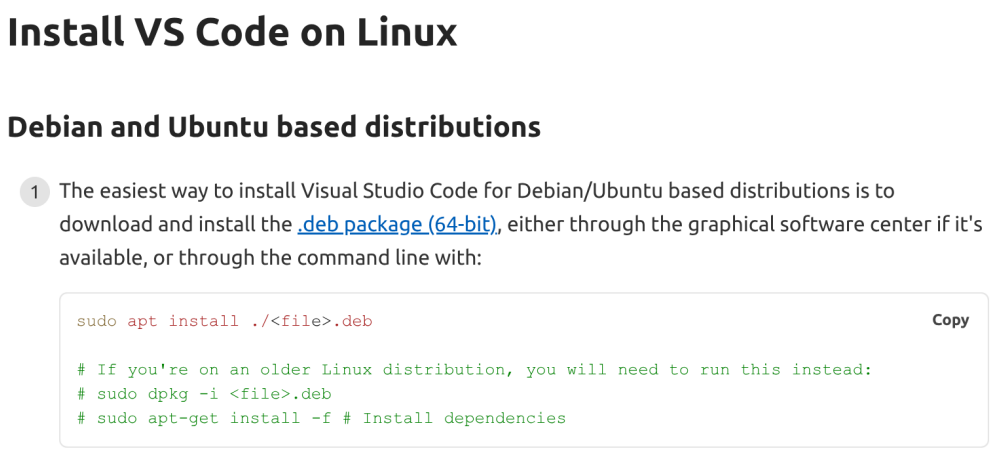


Figure 4.11: Installing Visual Studio Code

4.2.2 Turtlebot3 Setup

This engineering project uses the Turtlebot3 Waffle Pi version. The specific hardware configuration is shown in Table **??**. For the setup of the Single Board Computer (SBC) and OpenCR on Turtlebot3, please refer to the tutorials SBC Setup and OpenCR Setup.

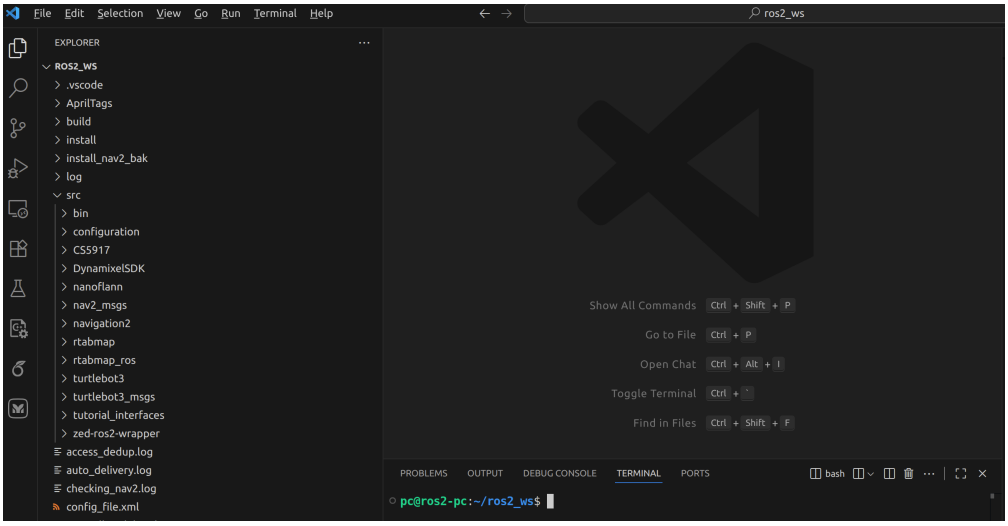


Figure 4.12: Opening ROS 2 Workspace

4.2.3 Jetson Orin Nano Setup

This engineering project uses the Jetson Orin Nano Developer Kit version. The specific hardware configuration is shown in Table **??**. On the Jetson Orin Nano, we mainly install the operating system, ROS 2, ZED SDK, ZED package, rtabmap package, and the engineering package `uoa_robot_drone`, and perform the corresponding environment configuration. Below, we will detail these steps.

Installing the Operating System

1. After registering a free account according to the tutorial Download and Run SDK Manager, use the SDK Manager to install Jet Pack version 6.2.1.

2. Access the graphical interface of the Jetson Orin Nano's Ubuntu system and connect to a wireless network.

3. To save resources, SSH into the Jetson Orin Nano and enter the command:

   ```
   ssh username@ip_address
   ```

   where `username` is the username of the Jetson Orin Nano, and `ip_address` is the IP address of the Jetson Orin Nano.

4. After entering the Ubuntu system of the Jetson Orin Nano, enter the command:

   ```
   sudo systemctl set-default multi-user.target
   sudo systemctl reboot
   ```

5. After rebooting, the Jetson Orin Nano will enter command line mode.

Installing ROS 2

This engineering project uses ROS 2 Humble Hawksbill as the version. Follow the tutorial Ubuntu (deb packages) to install step by step, and execute the steps shown in Figure 4.9.

Installing ZED SDK

This engineering project uses the CUDA 12 ZED SDK for Ubuntu 22 version 5.0. Follow the tutorial Download and Install the ZED SDK to install step by step. It is not recommended to install in `silent mode`.

Installing Packages

On the Jetson Orin Nano, the main packages used in this engineering project are the ZED 2 stereo camera, rtabmap, and the engineering package `uoa_robot_drone`. [On Jetson Orin Nano] The relevant installation commands are as follows:

```
$ source ~/.bashrc
$ mkdir -p ~/colcon_ws/src
$ cd ~/colcon_ws
$ git clone https://github.com/stereolabs/zed-ros2-wrapper.git src/zed-ros2-
    wrapper
$ git clone https://github.com/introlab/rtabmap.git src/rtabmap
$ git clone -b ros2 https://github.com/introlab/rtabmap_ros.git src/rtabmap_ros
$ git clone -b humble https://github.com/Jian-UOA/CS5917.git src/CS5917
$ sudo apt install python3-colcon-common-extensions
$ colcon build --symlink-install
```

Environment Configuration

[On Jetson Orin Nano] The relevant configuration commands are as follows:

```
$ echo 'source␣~/colcon_ws/install/setup.bash' >> ~/.bashrc
$ echo 'source␣/opt/ros/humble/setup.bash' >> ~/.bashrc
$ echo 'export␣ROS_DOMAIN_ID=30' >> ~/.bashrc
$ echo 'export␣RMW_IMPLEMENTATION=rmw_fastrtps_cpp' >> ~/.bashrc
$ source ~/.bashrc
```

Verifying Installation

To verify whether the installation on the Jetson Orin Nano was successful, follow these steps:

1. Verify that the ZED 2 camera is successfully installed

   (a) Open a terminal on the PC, SSH into the Jetson Orin Nano, and enter the command:

```
$ clear && cd ~/colcon_ws && ros2 launch
    zed_wrapper zed_camera.launch.py | tee zed.log
```

(b) If the ZED-related installation is successful, you should see output similar to Figure 4.13.

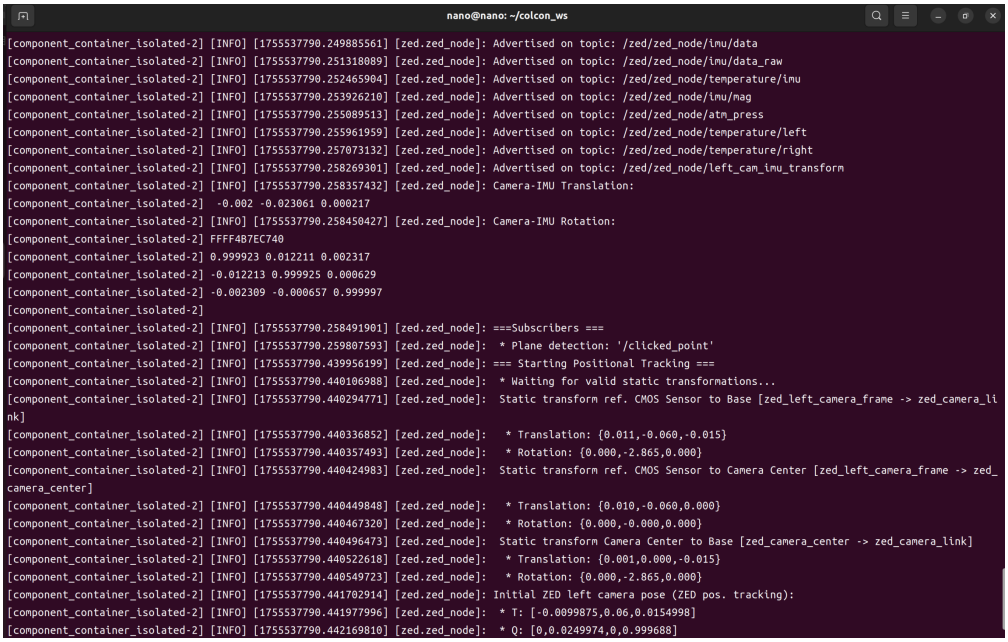


Figure 4.13: ZED Camera Launch Output

2. Verify that the rtabmap package is successfully installed

(a) On PC, press `Ctrl + Alt + T` to open a terminal, SSH into the Jetson Orin Nano, and enter the command:

```
$ clear && cd ~/colcon_ws && ros2 launch
    uoa_robot_drone orin_stereo_vslam.launch.py
    localization:=false | tee orin_vslam.log
```

(b) If the rtabmaprelated installation is successful, you should see output similar to Figure 4.14.

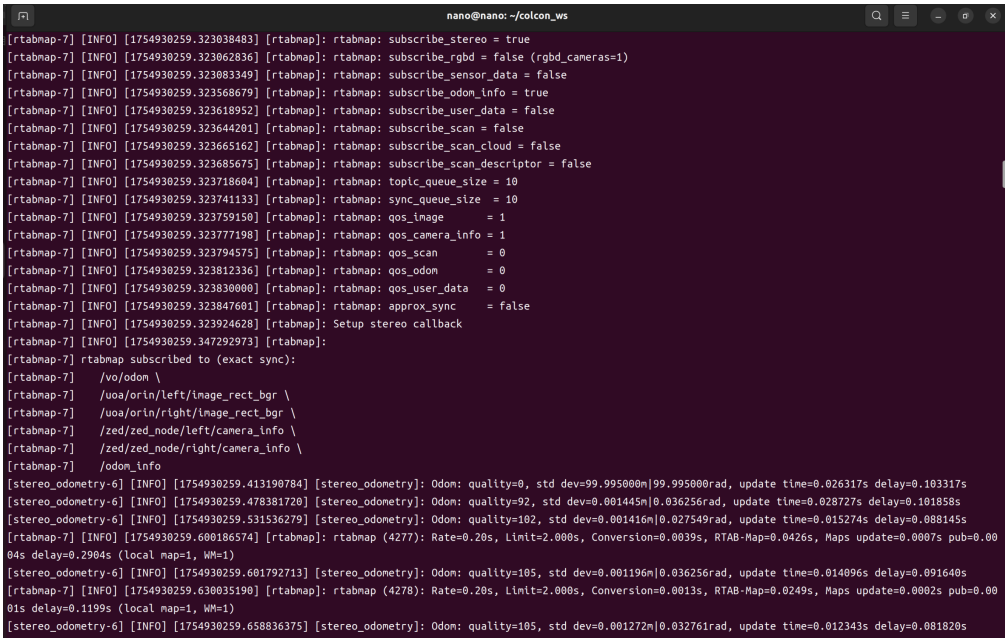


Figure 4.14: Orin Stereo Visual SLAM Launch Output

Chapter 5

Evaluation

Chapter 6

Conclusion

Bibliography

Angeli, A., Doncieux, S., Meyer, J., and Filliat, D. (2008a). Incremental vision-based topological SLAM. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1031–1036.

Angeli, A., Filliat, D., Doncieux, S., and Meyer, J.-A. (2008b). Fast and incremental method for loop-closure detection using bags of visual words. *IEEE Transactions on Robotics*, 24(5):1027–1037.

Bay, H., Ess, A., Tuytelaars, T., and Van Gool, L. (2008). Speeded up robust features (SURF). *Computer Vision and Image Understanding*, 110(3):346–359.

Botterill, T., Mills, S., and Green, R. (2011). Bag-of-words-driven, single-camera simultaneous localization and mapping. *Journal of Field Robotics*, 28(2):204–226.

Cummins, M. and Newman, P. (2008). FAB-MAP: probabilistic localization and mapping in the space of appearance. *The International Journal of Robotics Research*, 27(6):647–665.

Dissanayake, M. W. M. G., Newman, P., Clark, S., Durrant-Whyte, H. F., and Csorba, M. (2001). A solution to the simultaneous localization and map building (SLAM) problem. *IEEE Transactions on Robotics and Automation*, 17(3):229–241.

Konolige, K., Bowman, J., Chen, J., Mihelich, P., Calonder, M., Lepetit, V., and Fua, P. (2010). View-based maps. *The International Journal of Robotics Research*, 29(8):941–957.

Labbé, M. and Michaud, F. (2024). Memory management for real-time appearance-based loop closure detection. *arXiv preprint arXiv:2407.15890*.

Labbé, M. and Michaud, F. (2025). RTAB-Map: Real-time appearance-based mapping. Project website: https://introlab.github.io/rtabmap/. Accessed: 2025-08-14.

Muja, M. and Lowe, D. G. (2009). Fast approximate nearest neighbors with automatic algorithm configuration. In *Proceedings of the International Conference on Computer Vision Theory and Application (VISAPP)*, pages 331–340.

Newman, P., Cole, D., and Ho, K. (2006). Outdoor SLAM using visual appearance and laser ranging. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1180–1187.

Robotics, O. (2022). ROS 2 humble hawksbill: Release Notes. ROS 2 Documentation: https://docs.ros.org/en/humble/Releases/Release-Humble-Hawksbill.html. Accessed: 2025-08-16.

Sivic, J. and Zisserman, A. (2003). Video Google: A text retrieval approach to object matching in videos. In *Proceedings of the 9th International Conference on Computer Vision (ICCV)*, pages 1470–1478, Nice, France.

Appendix A

ROS Graphs

In ROS, the communication between nodes is facilitated through a system of topics, services, and actions. The ROS 2 graph is a representation of this communication network, showing how nodes are connected and how they exchange messages. Here are some ROS graphs from different stages of the engineering.

A.1 ROS Graph of the Mapping Stage

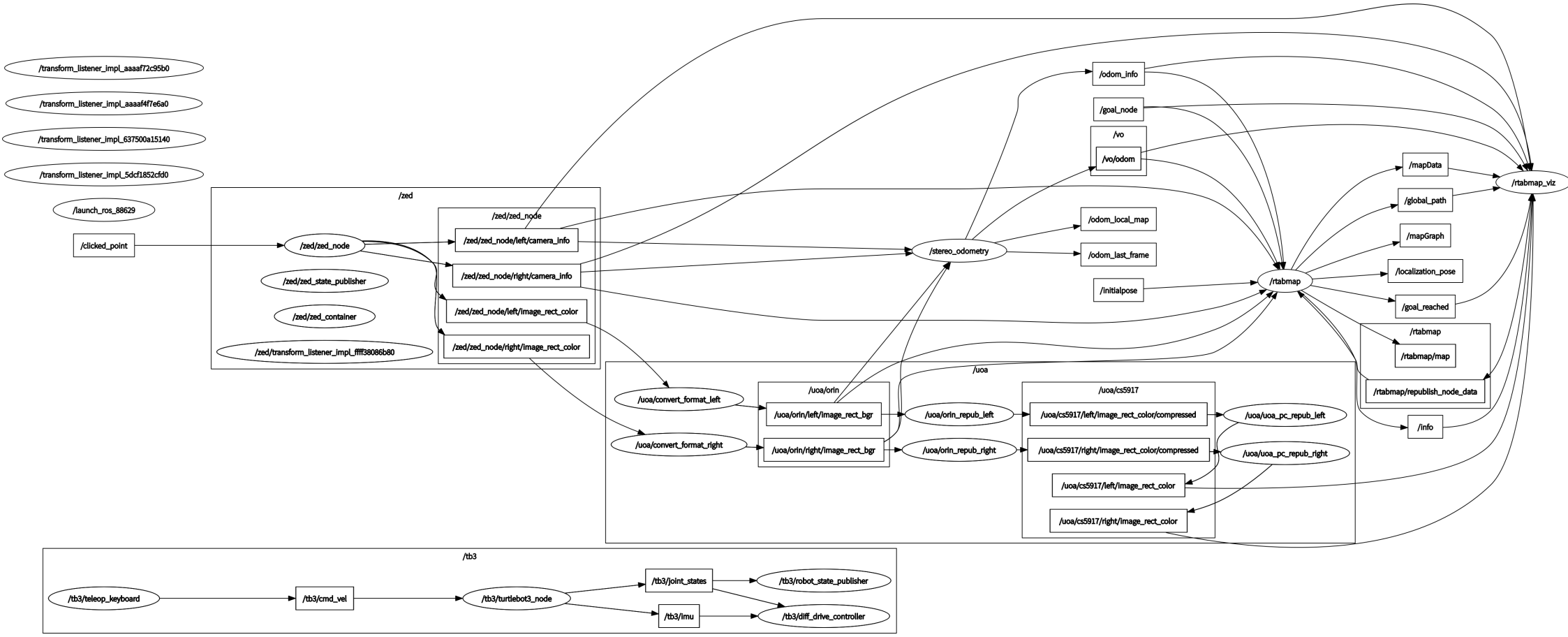


Figure A.1: ROS Graph of the Mapping Stage

A.2 ROS Graph of the Localization Stage

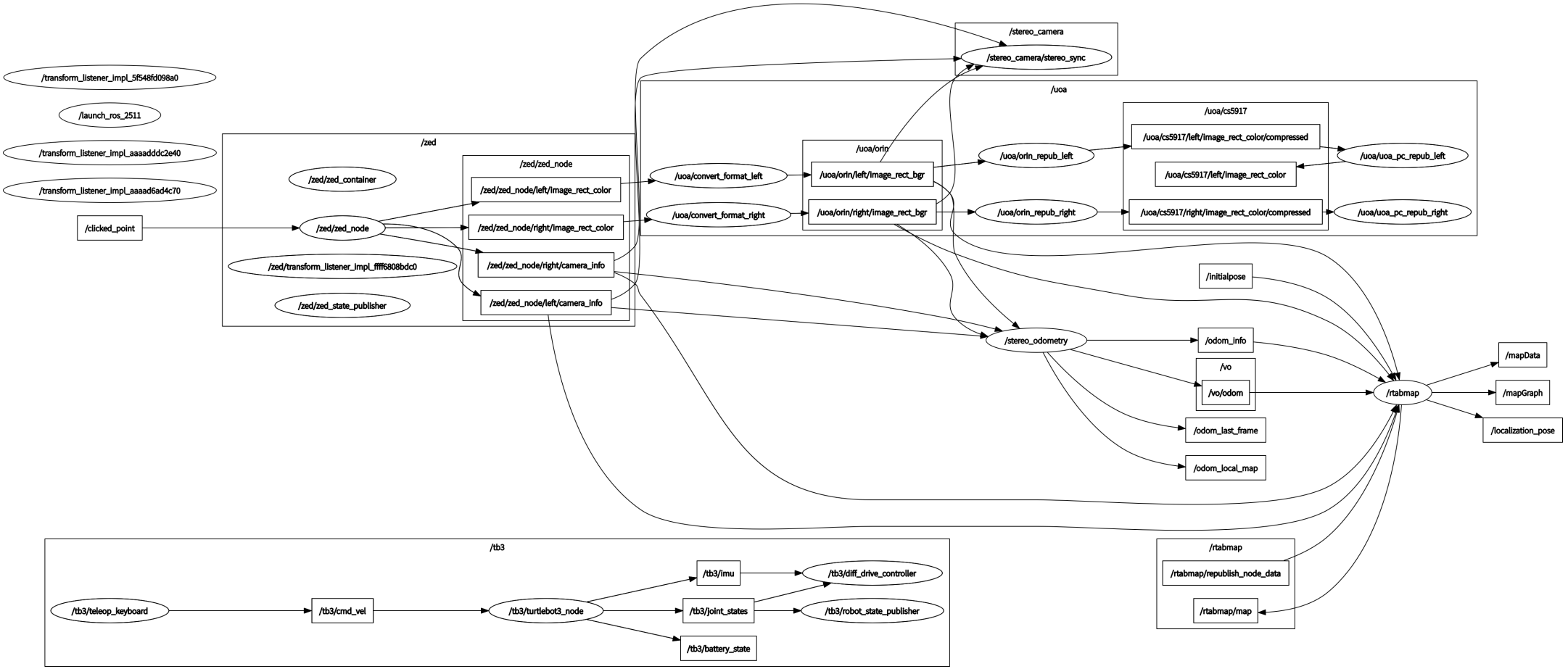


Figure A.2: ROS Graph of the Localization Stage

A.3 ROS Graph of the Navigation Stage

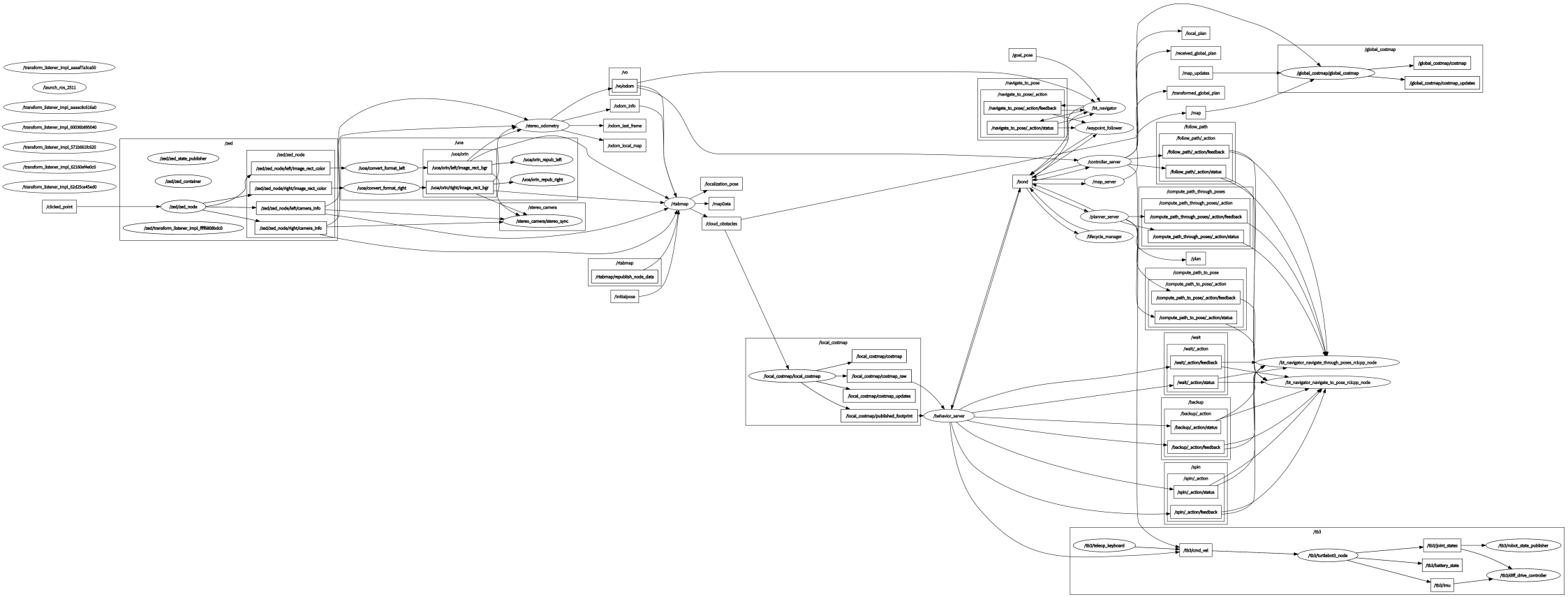


Figure A.3: ROS Graph of the Navigation Stage